

# WEBRUM-07: Session Replay

---

**Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 7 of 9 | **Created:** March 2026 | **Last Updated:** 07/08/2026

## Overview

---

Session Replay allows you to visually replay a user's interaction with your web application — seeing exactly what the user saw, where they clicked, how they scrolled, and when errors occurred. Unlike traditional analytics that show numbers, session replay provides qualitative context for quantitative data.

This notebook covers how Dynatrace Session Replay works (DOM snapshot recording vs. video), data privacy considerations, masking configuration, how to find and analyze replays via DQL, and strategies for correlating replay data with performance metrics. It also touches on integration patterns with third-party replay tools.

---

## Table of Contents

---

1. [How Session Replay Works](#)
  2. [Data Privacy and Masking](#)
  3. [Session Replay Configuration](#)
  4. [Finding Replay-Eligible Sessions](#)
  5. [Correlating Replay with Performance](#)
  6. [Identifying UX Issues via Replay](#)
  7. [Third-Party Replay Integration](#)
  8. [Summary and Next Steps](#)
-

# Prerequisites

---

| Requirement                   | Details                                                             |
|-------------------------------|---------------------------------------------------------------------|
| <b>Dynatrace Environment</b>  | SaaS with Session Replay license                                    |
| <b>Session Replay Enabled</b> | Configured for at least one web application                         |
| <b>Permissions</b>            | <code>storage:events:read</code> , Session Replay viewer permission |
| <b>Previous Notebooks</b>     | WEBRUM-01: RUM Fundamentals, WEBRUM-04: Session Analysis            |

## 1. How Session Replay Works

---

Dynatrace Session Replay captures the visual state of the web page as the user interacts with it. It does **not** record video — instead, it records DOM mutations:

### Recording Mechanism

1. **Initial Snapshot** — When a page loads, the RUM agent captures the full DOM structure and inline styles
2. **Mutation Recording** — As the user interacts, DOM mutations (element changes, attribute modifications, text updates) are recorded incrementally
3. **User Events** — Mouse movements, clicks, scrolls, and keyboard inputs are captured with timestamps
4. **Beacon Delivery** — Recorded data is sent to Dynatrace in compressed beacons
5. **Server-Side Reconstruction** — Dynatrace reconstructs the visual experience from the DOM mutations and events

## Recording Modes

| Mode             | Description                                     | Data Volume | Privacy Impact                     |
|------------------|-------------------------------------------------|-------------|------------------------------------|
| Visual replay    | Full DOM + CSS reconstruction                   | High        | Highest — captures visible content |
| Resource capture | Includes external resources (images, CSS files) | Very high   | Captures all visual assets         |
| Minimal mode     | DOM structure only, no external resources       | Low         | Lower — placeholder images         |

**Important:** Session Replay data is stored separately from RUM metrics and has its own retention period (typically 35 days by default in the classic model). Replay storage counts against your session replay quota.

**Update (SaaS 1.343, July 2026 — staged tenant rollout) — Session Replay reaches general availability on Grail.** Replay data is stored natively in the Grail data lakehouse ( `fetch user.replays` ), with the video player, timeline, and multi-platform support built on that storage. On Grail-based tenants, replay retention follows the Grail bucket configuration rather than the classic quota model above. The classic model remains current for tenants that have not yet received 1.343 or still store replay in the classic surface — verify which model your tenant uses before planning retention.

## 2. Data Privacy and Masking

Session Replay captures real user interactions, which may include sensitive data. Proper masking is essential for compliance with GDPR, CCPA, HIPAA, and other privacy regulations.

## Masking Levels

| Level           | Behavior                                   | Use Case                        |
|-----------------|--------------------------------------------|---------------------------------|
| Open            | No masking — all content visible           | Non-sensitive public pages only |
| Mask user input | Input field values replaced with asterisks | Default — protects typed data   |
| Mask all text   | All text content replaced with blocks      | Strict privacy environments     |
| Block element   | Entire elements replaced with placeholders | Specific sensitive components   |

## HTML Attributes for Granular Control

Apply masking directly in your HTML:

```
<!-- Mask specific input -->
<input type="text" data-dtrum-mask="true" />

<!-- Block entire section from replay -->
<div data-dtrum-block="true">
  <p>Sensitive content here</p>
</div>

<!-- Allow recording of specific element (override page-level masking) --
>
<span data-dtrum-unmask="true">Public content</span>
```

## CSS Selector Masking Rules

Configure in **Settings > Web and mobile monitoring > Session Replay > Data privacy**:

| Selector                | Effect                               |
|-------------------------|--------------------------------------|
| .credit-card-form input | Mask all inputs in credit card forms |
| [data-sensitive]        | Mask elements with custom attribute  |
| .pii-container          | Block entire PII containers          |

**Warning:** Always err on the side of over-masking. It is better to mask non-sensitive content than to accidentally capture PII. Conduct a privacy review before enabling Session Replay in production.

## 3. Session Replay Configuration

### Sampling Rate

Not every session needs to be recorded. Configure sampling to balance insight with cost:

| Scenario                     | Recommended Sample Rate |
|------------------------------|-------------------------|
| High-traffic production site | 1-5%                    |
| Internal application         | 10-25%                  |
| Pre-production testing       | 50-100%                 |
| Investigating specific issue | 100% (temporarily)      |

### Conditional Capture

Use capture rules to record specific session types:

- **Error-triggered replay** — Automatically capture replay when JavaScript errors occur
- **Conversion-path replay** — Capture sessions that reach checkout pages
- **Frustration-triggered replay** — Capture sessions with rage clicks or exit intent

### Storage Considerations

| Factor                     | Impact on Storage                                 |
|----------------------------|---------------------------------------------------|
| Page complexity (DOM size) | More complex pages = larger snapshots             |
| Session duration           | Longer sessions = more mutation data              |
| SPA vs. traditional        | SPAs generate more mutations from in-page updates |

| Factor                   | Impact on Storage                             |
|--------------------------|-----------------------------------------------|
| Resource capture enabled | Significantly increases storage (images, CSS) |

## 4. Finding Replay-Eligible Sessions

Use DQL to identify sessions that have replay data available, then navigate to them in the Dynatrace UI.

```
// Sessions with replay data available in the last 24 hours
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| filter isNotNull(hasSessionReplay) and hasSessionReplay == true
| summarize replay_sessions = count(),
    avg_actions = avg(userActionCount),
    avg_errors = avg(totalErrorCount),
    by:{application}
| sort replay_sessions desc
```

```
// Find replay sessions with errors – prioritize these for review
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| filter isNotNull(hasSessionReplay) and hasSessionReplay == true
| filter totalErrorCount > 0
| fieldsKeep sessionId, application, duration, userActionCount,
totalErrorCount, country, browserFamily
| sort totalErrorCount desc
| limit 20
```

```
// Replay coverage – what percentage of sessions have replay?
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| summarize total_sessions = count(),
    replay_sessions = countIf(isNotNull(hasSessionReplay) and
hasSessionReplay == true),
    by:{application}
| fieldsAdd replay_coverage_pct = round(toDouble(replay_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort total_sessions desc
```

## 5. Correlating Replay with Performance

Session Replay is most powerful when correlated with performance data. Use DQL to find sessions that experienced specific performance issues, then review the replay to understand the visual impact.

```
// Slow sessions with replay – sessions with poor page load times
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| filter isNotNull(hasSessionReplay) and hasSessionReplay == true
| fieldsAdd duration_sec = duration / 1s
| filter duration_sec > 120
| fieldsKeep sessionId, application, duration_sec, userActionCount,
totalErrorCount, country
| sort duration_sec desc
| limit 10
```

```
// Sessions with rage clicks that have replay – top candidates for UX review
fetch user.events, from:-24h
| filter type == "RageClick"
| summarize rage_clicks = count(), by:{sessionId, application}
| sort rage_clicks desc
| limit 10
```

**Tip:** Copy the `session.id` value from query results and paste it into the Dynatrace Session Replay search to jump directly to the replay.

## 6. Identifying UX Issues via Replay

Session Replay helps identify qualitative UX issues that metrics alone cannot reveal:

| UX Issue                    | What to Look For in Replay                                     |
|-----------------------------|----------------------------------------------------------------|
| <b>Confusing navigation</b> | Users going back and forth, visiting the same pages repeatedly |
| <b>Broken forms</b>         | Users attempting to submit forms multiple times                |
| <b>Invisible errors</b>     | JS errors that silently break functionality                    |

| UX Issue               | What to Look For in Replay                      |
|------------------------|-------------------------------------------------|
| Misleading UI elements | Users clicking on non-interactive elements      |
| Layout shifts          | Content jumping around during page load         |
| Slow feedback          | Long pauses after clicks before visual response |

## Replay Investigation Workflow

1. **Identify problematic sessions** via DQL (errors, rage clicks, abandonment)
2. **Watch the replay** in Dynatrace UI
3. **Correlate with waterfall** — Performance tab shows timing alongside replay
4. **Note the pattern** — Is this a one-off or a systematic issue?
5. **Quantify with DQL** — How many sessions are affected by this pattern?

```
// Abandoned sessions with replay – users who started but did not finish
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| filter isNotNull(hasSessionReplay) and hasSessionReplay == true
| filter userActionCount >= 3 and userActionCount <= 5
| filter totalErrorCount > 0
| fieldsKeep sessionId, application, userActionCount, totalErrorCount,
duration, country
| sort totalErrorCount desc
| limit 15
```

## 7. Third-Party Replay Integration

Some organizations use dedicated session replay tools alongside Dynatrace. Common integration patterns:

### Integration Approaches

| Tool                            | Integration Method               | Use Case                                    |
|---------------------------------|----------------------------------|---------------------------------------------|
| <b>Dynatrace Session Replay</b> | Built-in — no integration needed | Full-stack correlation (backend + frontend) |



| Tool             | Integration Method                          | Use Case                |
|------------------|---------------------------------------------|-------------------------|
| <b>LogRocket</b> | Session ID correlation via session property | Product analytics focus |
| <b>FullStory</b> | Session URL in custom session property      | UX research focus       |
| <b>Hotjar</b>    | Heatmaps + recordings complement RUM data   | Heatmap analysis        |

## Correlation Pattern

To link Dynatrace sessions with a third-party replay tool:

1. Capture the third-party session ID via the RUM JavaScript API:

```
// Example: send LogRocket session URL to Dynatrace
LogRocket.getSessionURL(function(sessionURL) {
  dtrum.sendSessionProperties(
    null, null,
    { "logrocket_url": sessionURL }
  );
});
```

2. Configure a custom string session property in Dynatrace for `logrocket_url`
3. Query sessions with the third-party URL for cross-referencing

**Note:** Dynatrace's native Session Replay has the advantage of being fully integrated with the performance waterfall, backend traces, and Dynatrace Intelligence — third-party tools typically lack this correlation.

## 8. Summary and Next Steps

In this notebook, we covered:

- **How Session Replay works** — DOM mutation recording, not video
- **Data privacy** — Masking levels, HTML attributes, CSS selector rules
- **Configuration** — Sampling rates, conditional capture, storage considerations

- **Finding replays** — DQL queries to locate replay-eligible sessions
- **Performance correlation** — Linking slow/error sessions to replay
- **UX issue identification** — Patterns to look for in replay analysis
- **Third-party integration** — Connecting Dynatrace with external replay tools

## Next Steps

- **WEBRUM-08: Dashboards and Alerting** — Build RUM dashboards with Apdex and set up performance alerts

## References

- [Session Replay](#)
- [Session Replay Privacy](#)
- [Session Replay Masking](#)

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*